Daniil Dorokhov

AI / LLM ENGINEER · PYTHON

Armenia (residence permit) · open to relocation · dorokhov.daniil69@gmail.com · t.me/ananasDDA · ananasdda.github.io/Portfolio · github.com/ananasDDA

SUMMARY

Engineer with four years in IT, specialising in putting large language and multimodal models into production. At R7 I embedded models into the testing pipeline: code generation under formal validation, vision-based screenshot analysis, and automated failure triage, with cost control and graceful degradation when a model is unavailable. Outside work: AI features in an app published on the App Store, and an iOS app for on-device LLM inference. I am building the classical ML foundation alongside this through computer science coursework at HSE, with machine learning and recommender systems in progress.

EXPERIENCE

QA Automation Lead, AI engineering in the test pipeline 02/2025 - present

R7, Corporate Server

Joined as the first QA engineer on the product; built the automation platform from scratch and embedded LLMs where a model does real work.

- **LLM code generation for automated tests** from TestIT cases: the prompt is assembled from case steps, Page Object sources, fixtures and reference tests in the module; the model's output is checked by AST parsing, import and fixture validation, and verification of every call against the real Page Object; on a validation error the request is retried automatically with a description of the problem. Test authoring: minutes instead of 15-30 minutes of manual work per case.
- **Vision analysis of visual regression, thousands of screenshots per run**: replaced manual review of pixel comparison results with a VLM pipeline - automatic assembly of reference/actual pairs with filename normalisation, a prompt carrying computer vision metrics (comparison score, contour counts) and test case context, and a structured JSON verdict with status, severity, confidence and a recommendation. When no reference exists, the pipeline degrades to analysing actual screenshots with reduced confidence; token cost is computed per session; only failed tests are analysed. I also prototyped an embeddings-based comparison variant.
- **Test result analysis service**: REST API and web interface, a pipeline from ingest to verdict - report parsing, metric enrichment, LLM analysis of run statistics, a quality gate with thresholds and a pass/fail verdict, and webhook notifications. Rate limiting and a request queue keep load and spend under control.
- **Automated failure triage in CI**: JUnit XML and log parsing, with the model returning a root cause and recommendation for every failed test, published to the GitHub step summary. Triage in seconds rather than hours; when a run is green, no model call is made at all.
- **AI analysis of installation logs** in Jenkins pipelines: secrets masked before the request, model selectable by parameter, model failure never affecting the pipeline result, and a designed path to on-premise inference.
- **Platform prototype on Jupyter and Papermill**: a five-notebook pipeline (repository analysis, test scenario parsing, adaptation of tests to four OS and browser configurations, AI analysis of PDF reports, documentation generation), executed programmatically through Papermill and the Jupyter Server REST API, deployed in Docker on a VPS. Once proven, it was migrated into production Jenkins and GitHub Actions pipelines.
- The engineering foundation around this: about 400 automated tests in Playwright and pytest, Infrastructure as Code, 11 servers across 5 Linux families.

Python · LLM APIs · Jupyter · Papermill · Playwright · pytest · Jenkins · GitHub Actions · Docker · Linux

QA Automation Engineer 03/2024 - 02/2025

Avroid, startup

- **RAG over the Aurora OS documentation plus an agent** for the support team, delivered as one project: an answer grounded in the documentation together with automatic reproduction of the scenario on a mobile Linux emulator.
- Testing process from scratch, automation in Python, and a Telegram bot integrated with the CRM.

QA Engineer, release acceptance 08/2022 - 03/2024

MyOffice

Acceptance of B2B products, UI and API automation (Selenium, Appium, requests), and six months covering DevOps work (Docker Compose, mock servers, Jenkins).

AI PROJECTS

PRO Local AI Lab

On-device LLM and VLM inference on iOS

- An iOS app that runs language and multimodal models directly on the iPhone, with no cloud and no subscription.
- Inference: MLX, llama.cpp (GGUF), Core ML, Apple Intelligence; models pulled from Hugging Face.
- Multimodality: text, images (VLM), speech recognition (Whisper), speech synthesis (TTS).
- Automatic model selection for the device hardware; metrics: tokens per second, latency, memory footprint.

Swift · MLX · llama.cpp · Core ML · Hugging Face

P.R.O.

AI-powered sports app, published on the App Store

Backend and AI features: personalised recommendations, goal setting matched to fitness level, progress forecasting, and an athlete assistant. Taken from idea to publication.

SKILLS

Languages: Python (primary), Swift, Kotlin, Java, Bash, SQL

LLM and VLM: API integration, RAG, agentic workflows, prompt engineering with guardrails, validation of model output (AST, verification against the codebase), vision analysis, call cost control, graceful degradation

On-device inference: MLX, llama.cpp, Core ML, Hugging Face, Whisper, TTS; quality, speed and memory trade-offs

Data and experimentation: pandas, numpy, Jupyter, Papermill

Infrastructure: Docker, Kubernetes, AWS, Jenkins, GitLab CI, GitHub Actions, REST API, Kafka, Sentry, Grafana, Git, Linux

Languages: Russian - native; English - Upper-Intermediate (B2)

Mathematics and CS (HSE): calculus, discrete mathematics, algorithms and data structures, computer architecture and operating systems; machine learning and recommender systems in progress

EDUCATION

BSc, Business Informatics

Plekhanov Russian University of Economics

Auditing student

HSE University, Faculty of Computer Science